ToyPlay — คู่มือเตรียมตอบคำถามอาจารย์

เอกสารนี้รวบรวมคำถามที่อาจารย์มีแนวโน้มจะถามเกี่ยวกับระบบ ToyPlay
พร้อมคำตอบที่อ้างอิงจากโค้ดจริงในโปรเจกต์ ใช้ทบทวนก่อนนำเสนอ

ตัวเลขและข้อเท็จจริงที่ควรจำ

- ชื่อระบบ: ToyPlay — เว็บแอปพลิเคชันร้านของเล่นออนไลน์ (E-Commerce)
- เฟรมเวิร์ก: Django 6.0.6 + Django REST Framework
- ฐานข้อมูล: SQLite (ตอนพัฒนา) / PostgreSQL (ตอนอยู่บน Render)
- หน้าบ้าน: HTML + Bootstrap 5.3.3 + CSS เองที่ static/css/style.css
- ผู้ใช้ 3 บทบาท: สมาชิก (member), พนักงาน (staff), ผู้ดูแลระบบ (admin)
- จำนวนตาราง/โมเดล: 11 โมเดล (Category, Product, Review, Wishlist, Coupon, Cart, CartItem, Order, OrderItem, UserProfile, EmployeeProfile)
- Primary Key ของทุกตาราง: ฟิลด์ id อัตโนมัติ (BigAutoField)
- จำนวนเทสต์อัตโนมัติ: 21 รายการ ผ่านทั้งหมด
- สินค้าต่อหน้า: 6 ชิ้น (หน้าเว็บ) / API แบ่งหน้าละ 10 รายการ
- บัญชีผู้ดูแลระบบสาธิต: username boss รหัสผ่าน 1
- คูปองสาธิต: WELCOME10 (ลด 10% ไม่มีขั้นต่ำ), TOY20 (ลด 20% ขั้นต่ำ 1,000 บาท จำกัด 100 ครั้ง)

A. คำถามภาพรวมและแนะนำตัวระบบ

ถาม: ขอให้อธิบายระบบของคุณสั้น ๆ ว่าทำอะไรได้บ้าง

- **ตอบสั้น:** เป็นเว็บร้านขายของเล่นออนไลน์ที่ทำครบวงจร ตั้งแต่สมัครสมาชิก เลือกดูสินค้าตามหมวดหมู่ ค้นหา ใส่ตะกร้า ใช้คูปอง สั่งซื้อ ให้คะแนนรีวิว บันทึกรายการโปรด ติดตามสถานะคำสั่งซื้อ ไปจนถึงระบบหลังบ้านให้พนักงานและแอดมินจัดการสินค้า สมาชิก และออเดอร์ พร้อมมี REST API ให้ต่อยอด
- **ขยาย:** ฝั่งลูกค้าเป็นหน้าบ้าน (Storefront) ฝั่งร้านเป็น Dashboard หลังบ้าน และมี Django Admin สำหรับจัดการข้อมูลทุกตาราง

ถาม: ทำไมเลือกใช้ Django ไม่ใช้ Flask หรือภาษาอื่น

- **ตอบสั้น:** เพราะ Django มีมาให้ครบแบบ "batteries included" ทั้ง ORM, ระบบล็อกอิน/สิทธิ์ผู้ใช้, Django Admin และระบบฟอร์ม/validation ทำให้พัฒนาระบบ CRUD และระบบสมาชิกได้เร็วและปลอดภัยกว่า
- **ขยาย:** Flask ต้องประกอบชิ้นส่วนเองหลายอย่าง งานนี้เป็นระบบฐานข้อมูลที่มีหลายตารางและหลายบทบาท Django จึงเหมาะกว่า

ถาม: ระบบนี้ใครเป็นผู้ใช้ และแต่ละคนต่างกันอย่างไร

- **ตอบสั้น:** มี 3 บทบาท คือ สมาชิกซื้อสินค้าได้ พนักงานจัดการสินค้า/ออเดอร์ได้ และแอดมินจัดการได้ทุกอย่างรวมถึงบัญชีผู้ใช้

B. สถาปัตยกรรมและพื้นฐาน Django

ถาม: Django ทำงานแบบ MVC หรือ MTV อธิบายให้ฟัง

- **ตอบสั้น:** Django ใช้สถาปัตยกรรม MTV (Model–Template–View) โดย Model คือโครงสร้างฐานข้อมูล, View คือตรรกะประมวลผล และ Template คือหน้า HTML ที่แสดงผล

- **ขยาย:** เทียบกับ MVC แล้ว View ของ Django ทำหน้าทีเหมือน Controller ส่วน Template ทำหน้าทีเหมือน View ใน MVC ส่วน Model เหมือนกันทังสองแบบ และมี URL Router คอยจับคู่ URL ไปยัง View

ถาม: ลำดับการทำงานเมือผู้ใช้เปิดหน้าเว็บหนึงหน้าเป็นอย่างไร

- **ตอบสัน:** เบราว์เซอร์ส่งคำขอเข้า URL Router → Router เรียก View ทีเกียวข้อง → View คุยฐานข้อมูลผ่าน Model → ส่งข้อมูลไปเรนเดอร์ใน Template → ได้ HTML กลับไปทีเบราว์เซอร์

ถาม: ไฟล์ urls.py ทำหน้าทีอะไร แยกเป็นสองไฟล์ทำไม

- **ตอบสัน:** myproject/urls.py เป็นเส้นทางหลักของทังเว็บ ส่วน myapp/api_urls.py รวมเส้นทางของ REST API แยกออกจากกันเพือให้อ่านง่ายและดูแลสะดวก

C. ฐานข้อมูลและโมเดล (จุดทีอาจารย์ชอบถาม)

ถาม: Primary Key ของตารางคุณคืออะไร

- **ตอบสัน:** ทุกตารางใช้ Primary Key ชือ id ที Django สร้างให้อัตโนมัติแบบเพิมค่าเอง (auto-increment) ชนิด BigAutoField
- **ขยาย:** ฟิลด์ที unique เช่น Category.name, Coupon.code หรือคู่ unique_together อย่าง (product, user) ใน Review และ (user, product) ใน Wishlist ยังไม่ใช่ Primary Key — Primary Key ยังเป็น id เหมือนเดิม ถ้าจะเปลียนต้องระบุ primary_key=True ในฟิลด์นัน ซึงตอนนียังไม่ได้ทำ

ถาม: ขอให้อธิบายความสัมพันธ์ระหว่างตาราง

- **ตอบสัน:** มี One-to-One (ผู้ใช้กับโปรไฟล์, ผู้ใช้กับตะกร้า), One-to-Many (ผู้ใช้กับคำสังซือ, คำสังซือกับรายการสินค้า) และ Many-to-One (สินค้ากับหมวดหมู่)
- **ขยาย:** UserProfile, EmployeeProfile และ Cart เป็น OneToOneField ไปยัง User; Product มี ForeignKey ไปยัง Category; Order มี ForeignKey ไปยัง User; OrderItem มี ForeignKey ไปยัง Order; Review และ Wishlist อ้างทัง Product และ User

ถาม: ทำไม OrderItem ต้องเก็บ product_name และ price ซ้ำ ทังทีมี FK ไป Product แล้ว

- **ตอบสัน:** เพือเก็บข้อมูล ณ เวลาทีสังซือ ถ้าภายหลังราคาสินค้าเปลียนหรือสินค้าถูกลบ ประวัติคำสังซือเดิมยังคงถูกต้อง
- **ขยาย:** FK ไป Product ตัง on_delete เป็น SET_NULL เพือไม่ให้ประวัติออเดอร์หายเมือลบสินค้า

ถาม: on_delete มีแบบไหนบ้าง ใช้ต่างกันอย่างไร

- **ตอบสัน:** ทีใช้หลัก ๆ คือ CASCADE คือลบแม่แล้วลบลูกตาม และ SET_NULL คือลบแม่แล้วตังค่า FK เป็น null
- **ขยาย:** เช่น ลบ Category แล้ว Product ไม่หายแต่ category เป็น null (SET_NULL) แต่ลบ Order แล้ว OrderItem หายตาม (CASCADE)

ถาม: unique_together ในรีวิวและรายการโปรดมีไว้ทำไม

- **ตอบสัน:** เพือกันข้อมูลซ้ำ คือ 1 ผู้ใช้รีวิวสินค้าเดิมได้ครังเดียว และ 1 ผู้ใช้กดหัวใจสินค้าเดิมซ้ำไม่ได้

ถาม: คุณสร้างโปรไฟล์และตะกร้าอัตโนมัติได้อย่างไร

- **ตอบสัน:** ใช้ signal post_save ของโมเดล User เมือสร้างผู้ใช้ใหม่ ระบบจะสร้าง UserProfile และ Cart ให้ทันที
- **ขยาย:** อยู่ใน myapp/models.py ฟังก์ชัน ensure_user_profile_and_cart ทำให้ทุกบัญชีใช้งานได้ทันทีไม่ต้องสร้างโปรไฟล์เอง

D. ระบบสมาชิกและสิทธิ์การเข้าถึง

ถาม: ระบบล็อกอินทำงานอย่างไร รหัสผ่านเก็บแบบไหน

- **ตอบสั้น:** ใช้ระบบ auth ของ Django โดยสมัครผ่าน RegisterForm รหัสผ่านถูกเข้ารหัสด้วย algorithm PBKDF2 ผ่าน set_password ไม่ได้เก็บเป็นข้อความธรรมดา
- **ขยาย:** เข้าสู่ระบบใช้ LoginView ของ Django และออกจากระบบผ่าน view ที่เขียนเองเพื่อเคลียร์ session

ถาม: คุณจำกัดสิทธิ์การเข้าถึงอย่างไร พนักงานกับแอดมินต่างกันตรงไหน

- **ตอบสั้น:** ใช้ decorator @login_required และ @user_passes_test โดยเช็ก is_staff สำหรับพนักงาน และ is_superuser สำหรับแอดมิน
- **ขยาย:** พนักงาน (is_staff=True) เข้า Dashboard จัดการสินค้าและออเดอร์ได้
  ส่วนการจัดการบัญชีผู้ใช้และการกำหนดสิทธิ์จำกัดเฉพาะแอดมิน (is_superuser=True) ผ่านฟังก์ชัน is_admin_user

ถาม: ถ้าสมาชิกธรรมดาพิมพ์ URL หน้า Dashboard ตรง ๆ จะเกิดอะไร

- **ตอบสั้น:** ระบบจะเช็กสิทธิ์แล้ว redirect ไปหน้า login เพราะไม่ผ่านเงื่อนไข user_passes_test

ถาม: แอดมินถูกสร้างขึ้นมาอย่างไร ทำไมไม่ต้องสร้างมือ

- **ตอบสั้น:** มีสองทาง ทางแรก migration 0009 สร้างบัญชีเริ่มต้น boss ให้ และทางที่สองใช้ Environment Variables DJANGO_SUPERUSER_USERNAME/PASSWORD ที่จะสร้างหรืออัปเดตแอดมินอัตโนมัติทุกครั้งที่รัน migrate ผ่าน signal post_migrate
- **ขยาย:** การใช้ environment variables ทำให้บน Production ไม่ต้องฝังรหัสผ่านในโค้ด

ถาม: ทำไมไม่ทำโมเดล User เอง (Custom User Model)

- **ตอบสั้น:** โครงงานใช้ User มาตรฐานของ Django แล้วเสริมข้อมูลด้วย UserProfile และ EmployeeProfile แบบ One-to-One ซึ่งเพียงพอต่อความต้องการ
- **ขยาย:** ข้อดีคือได้ระบบ auth, admin, สิทธิ์ มาครบ ถ้าเริ่มใหม่จริงจังควรใช้ Custom User ตั้งแต่ต้น แต่ project นี้เลือกวิธีที่เข้าใจง่ายกว่า

E. ฟีเจอร์ฝั่งลูกค้า

ถาม: การค้นหาสินค้าทำงานอย่างไร ปลอดภัยจาก SQL Injection ไหม

- **ตอบสั้น:** ค้นหาจากชื่อและรายละเอียดด้วย Q objects และ icontains ซึ่ง Django ORM แปลงเป็น SQL แบบมี parameter กำกับ จึงกัน SQL Injection ได้
- **ขยาย:** ยังกรองตามหมวดหมู่ด้วย category และเรียงตามราคา/ความใหม่ด้วยพารามิเตอร์ sort

ถาม: Pagination ทำอย่างไร

- **ตอบสั้น:** ใช้ Django Paginator แบ่งหน้าละ 6 ชิ้นสำหรับหน้าเว็บ และตั้ง PAGE_SIZE = 10 สำหรับ REST API

ถาม: ตะกร้าสินค้าทำงานอย่างไร 1 คนมีกี่ตะกร้า

- **ตอบสั้น:** 1 ผู้ใช้มี 1 ตะกร้า (OneToOne) ในตะกร้ามี CartItem หลายรายการ
  แต่ละรายการเก็บสินค้าและจำนวน
- **ขยาย:** การเพิ่มตรวจสต็อกก่อน ถ้าสินค้าหมดหรือสั่งเกินจำนวนที่มี ระบบจะแจ้งเตือนและไม่เพิ่ม

ถาม: ตอนกดชำระเงิน (checkout) ระบบทำอะไรบ้าง

- **ตอบสั้น:** ตรวจตะกร้าว่าง ตรวจสต็อกทุกชิ้น ตรวจคูปอง คำนวณส่วนลด สร้างคำสั่งซื้อและรายการสินค้า ตัดสต็อก เพิ่มจำนวนการใช้คูปอง แล้วล้างตะกร้า ทั้งหมดทำงานใน transaction เดียว
- **ขยาย:** ใช้ transaction.atomic เพื่อให้ถ้ามีข้อผิดพลาดกลางทาง ข้อมูลจะไม่ถูกบันทึกครึ่ง ๆ

ถาม: คูปองส่วนลดมีเงื่อนไขอะไรบ้าง

- **ตอบสั้น:** เป็นส่วนลดแบบเปอร์เซ็นต์ มีเงื่อนไขเปิดใช้งาน ต้องไม่หมดอายุ ยอดสั่งซื้อถึงขั้นต่ำ และยังไม่ใช้ครบจำนวนครั้งที่จำกัด

ถาม: สต็อกตัดยังไงให้ไม่ติดลบ

- **ตอบสั้น:** ตอนสั่งซื้อจะเซ็ตสต็อกใหม่เป็นค่ามากกว่าระหว่าง stock ลบ quantity กับ 0 และก่อนหน้านั้นก็ตรวจแล้วว่าสั่งไม่เกินสต็อก

ถาม: รีวิวเป็นอย่างไร 1 คนรีวิวได้กี่ครั้ง

- **ตอบสั้น:** ให้คะแนน 1–5 ดาวพร้อมความคิดเห็น 1 ผู้ใช้รีวิวสินค้าเดิมได้ครั้งเดียวและแก้ไขได้ โดยใช้ update_or_create

ถาม: รายการโปรด (Wishlist) ทำงานอย่างไร

- **ตอบสั้น:** กดหัวใจเพื่อเพิ่ม ถ้ากดซ้ำจะลบออก ใช้ get_or_create ตรวจว่ามีอยู่แล้วหรือยัง

ถาม: การติดตามสถานะคำสั่งซื้อแสดงอย่างไร

- **ตอบสั้น:** แสดงเป็น timeline ตามลำดับ รอดำเนินการ → ยืนยันคำสั่งซื้อ → กำลังจัดส่ง → สำเร็จ และมีสถานะยกเลิกแยกออกมา คำนวณตำแหน่งปัจจุบันด้วยเมธอด tracking_index

F. ระบบหลังบ้าน (Dashboard และ Admin)

ถาม: Dashboard หลังบ้านมีอะไรบ้าง

- **ตอบสั้น:** มีสถิติยอดขายรวม จำนวนคำสั่งซื้อ คำสั่งซื้อรอดำเนินการ จำนวนสมาชิก จำนวนสินค้า สินค้าใกล้หมด สินค้าขายดี และคำสั่งซื้อล่าสุด พร้อมตารางจัดการสินค้าและสมาชิก

ถาม: สถิติคำนวณอย่างไร

- **ตอบสั้น:** ใช้ aggregate ของ ORM เช่น Sum ยอดขาย, Count นับจำนวน และ Avg หาคะแนนรีวิว โดยยอดขายไม่รวมออเดอร์ที่ถูกยกเลิก

ถาม: สินค้าใกล้หมดนิยามว่าอะไร

- **ตอบสั้น:** สต็อกน้อยกว่าหรือเท่ากับ 5 ชิ้น และมี property is_low_stock ช่วยตรวจสอบ

ถาม: จัดการคำสั่งซื้อทำอะไรได้

- **ตอบสั้น:** ค้นหาด้วยชื่อผู้ใช้หรือเลขที่ออเดอร์ กรองตามสถานะ และอัปเดตสถานะของออเดอร์ได้

ถาม: Django Admin ต่างจาก Dashboard ที่ทำเองยังไง

- **ตอบสั้น:** Django Admin เป็นหน้าจัดการข้อมูลทุกตารางที่ Django ให้มาฟรี ส่วน Dashboard ที่ทำเองออกแบบมาให้พนักงานใช้งานง่ายและดูสถิติได้

G. REST API

ถาม: ทำไมต้องมี REST API

- **ตอบสั้น:** เพื่อให้ระบบอื่นหรือแอปมือถือในอนาคตดึงข้อมูลสินค้า ตะกร้า และคำสั่งซื้อไปใช้ต่อได้ โดยไม่ผูกกับหน้าเว็บ

ถาม: API ยืนยันตัวตนอย่างไร

- **ตอบสั้น:** ใช้ Token Authentication ของ DRF สำหรับไคลเอนต์ภายนอก และรองรับ Session Authentication สำหรับผู้ใช้ที่ล็อกอินในเว็บอยู่แล้ว
- **ขยาย:** ขอ token ได้ที่ /api/auth/login/ แล้วส่งมาใน header แบบ Authorization: Token ...

ถาม: API จำกัดสิทธิ์อย่างไร

- **ตอบสั้น:** ใช้ permission class IsStaffOrReadOnly คืออ่านได้ทุกคน แต่เพิ่ม/แก้/ลบสินค้าได้เฉพาะ staff

ถาม: API endpoint หลัก ๆ มีอะไรบ้าง

- **ตอบสั้น:** /api/products/ จัดการสินค้า, /api/auth/register/ /api/auth/login/ /api/auth/me/ เกี่ยวกับผู้ใช้, /api/cart/ และ /api/cart/items/<id>/ เกี่ยวกับตะกร้า และ /api/orders/ สำหรับคำสั่งซื้อ

ถาม: Serializer ทำหน้าที่อะไร

- **ตอบสั้น:** แปลงข้อมูลจากโมเดลเป็น JSON เพื่อส่งออก และแปลง JSON ขากลับเป็นข้อมูลเพื่อบันทึก พร้อมกำหนดฟิลด์ที่อ่านได้อย่างเดียว

H. Deployment และความปลอดภัย

ถาม: เอาเว็บขึ้นออนไลน์ได้อย่างไร

- **ตอบสั้น:** Deploy บน Render โดยเชื่อมกับ GitHub เมื่อ push ขึ้น branch main จะ deploy อัตโนมัติ ใช้ Gunicorn เป็นเซิร์ฟเวอร์

ถาม: Build Command กับ Start Command คืออะไร

- **ตอบสั้น:** Build ติดตั้งไลบรารีและรัน collectstatic กับ migrate ส่วน Start รัน Gunicorn ชี้ไปที่ myproject.wsgi:application

ถาม: ต้องตั้งค่า Environment Variables อะไรบน Production

- **ตอบสั้น:** SECRET_KEY, DEBUG=False, ALLOWED_HOSTS, DATABASE_URL และ DJANGO_SUPERUSER_USERNAME/PASSWORD สำหรับสร้างแอดมิน

ถาม: ไฟล์ static และรูปที่อัปโหลดจัดการอย่างไร

- **ตอบสั้น:** ไฟล์ static เสิร์ฟผ่าน WhiteNoise ส่วนรูปสินค้าและรูปโปรไฟล์เก็บในโฟลเดอร์ media และใช้ Pillow จัดการรูป

ถาม: เรื่องความปลอดภัยทำอะไรไว้บ้าง

- **ตอบสั้น:** รหัสผ่านเข้ารหัส ไม่เก็บ plain text, ใช้ CSRF protection ของ Django, สิทธิ์ผู้ใช้แยกตามบทบาท, ปิด DEBUG บน Production และเก็บ SECRET_KEY ใน environment variable

ถาม: DEBUG และ SECRET_KEY ถ้าไม่ได้ตั้งค่าจะเป็นอะไร

- **ตอบสั้น:** ค่าเริ่มต้นในโค้ดคือ DEBUG = True และ SECRET_KEY เป็นข้อความ dev ชั่วคราว ซึ่งใช้ได้แค่ตอนพัฒนา บน Production ต้องตั้งใน environment variables เสมอ

I. การทดสอบ

ถาม: ทดสอบระบบอย่างไรบ้าง

- **ตอบสั้น:** ใช้ Django TestCase เขียนเทสต์อัตโนมัติ 21 รายการ ครอบคลุมการสมัคร/ล็อกอิน สิทธิ์ การ CRUD สินค้า การค้นหา/แบ่งหน้า หมวดหมู่ รีวิว รายการโปรด คูปอง การตัดสต็อก และการสร้างแอดมินจาก environment

ถาม: รันเทสต์ยังไง และผลเป็นอย่างไร

- **ตอบสั้น:** รันคำสั่ง python manage.py test ผลคือผ่านทั้งหมด 21 รายการ และ python manage.py check ไม่พบข้อผิดพลาด

ถาม: มีเทสต์ที่ทดสอบเรื่องสิทธิ์ไหม

- **ตอบสั้น:** มี ทดสอบว่าสมาชิกธรรมดาเข้า Dashboard ไม่ได้ และทดสอบว่าแอดมินถูกสร้างจาก environment variable ได้จริง

J. คำถามเชิงลึกและกับดัก (เตรียมตอบแบบซื่อสัตย์)

ถาม: ตอนตัดสต็อกถ้ามีคนสั่งพร้อมกันจะเกิดปัญหาไหม

- **ตอบสั้น:** ในระบบปัจจุบันใช้การอ่านค่ามาแล้วลบและบันทึกกลับ ซึ่งเรียกว่า read-modify-write หากมีผู้ใช้กดสั่งพร้อมกันมาก ๆ ทฤษฎีแล้วอาจเกิดสต็อกผิดพลาดได้ (race condition)
- **ขยาย/ทางแก้:** แนวทางที่ถูกคือใช้ F() expression ให้ฐานข้อมูลลบเองแบบ atomic หรือใช้ select_for_update() ล็อกแถวนั้นระหว่างทำรายการ ปัจจุบันยังไม่ได้ทำ
  จุดนี้ถือเป็นข้อจำกัดที่รู้ตัวและมีแนวทางแก้

ถาม: คูปองแบบจำกัดจำนวนครั้งมีปัญหาเดียวกันไหม

- **ตอบสั้น:** ใช่ จำนวนครั้งที่ใช้ (used_count) ก็มีโอกาสเกิด race condition แบบเดียวกัน แนวทางแก้คือใช้ F() หรือล็อกแถวระหว่างนับ

ถาม: API ตั้ง permission เริ่มต้นเป็นอะไร

- **ตอบสั้น:** ค่า default ของ DRF ในโปรเจกต์ตั้งเป็น AllowAny แต่ที่ endpoint ที่ต้องล็อกอินได้ระบุ IsAuthenticated ชัดเจน และการเขียนสินค้าคุมด้วย IsStaffOrReadOnly
- **ขยาย:** ถ้าจะให้รัดกุมขึ้น ควรตั้ง default เป็น IsAuthenticated แล้วเปิด AllowAny เฉพาะจุด

ถาม: ทำไมใช้ built-in User ไม่ใช้ Custom User

- ดูข้อ D. ทำไมไม่ทำโมเดล User เอง (ตอบเรื่อง trade-off และถ้าเริ่มใหม่ควรทำตั้งแต่ต้น)

ถาม: การค้นหาใช้ LIKE ธรรมดา ถ้าข้อมูลเยอะจะช้าไหม

- **ตอบสั้น:** icontains เทียบเท่า LIKE ที่ไม่มี index จึงช้ากว่าเมื่อข้อมูลมาก แนวทางพัฒนาคือเพิ่มดัชนีหรือใช้ full-text search ในอนาคต

ถาม: serve ไฟล์ media ใน Production ปลอดภัยไหม

- **ตอบสั้น:** ปัจจุบันเสิร์ฟผ่าน Django ซึ่งไม่เหมาะกับงานจริง งาน production ควรใช้ Cloud Storage หรือ reverse proxy เสิร์ฟไฟล์แทน

K. ข้อจำกัดและแนวทางพัฒนา

ถาม: ระบบมีข้อจำกัดอะไรบ้าง

- **ตอบสั้น:** ข้อมูลและไฟล์รูปบนแพลนฟรีของ Render ไม่ถาวรเมื่อ redeploy ยังไม่มีระบบชำระเงินจริง ยังไม่รองรับหลายภาษา และมีประเด็น race condition ของสต็อก/คูปองที่กล่าวมา

ถาม: ถ้ามีเวลาพัฒนาต่อจะทำอะไร

- **ตอบสั้น:** เชื่อม PostgreSQL ถาวรและเก็บไฟล์บน Cloud Storage เพิ่มชำระเงินออนไลน์ เพิ่มการแจ้งเตือนอีเมล เพิ่มเอกสาร API อัตโนมัติ และทำ CI/CD

L. คำถามสั้นที่ควรตอบได้ทันที

- **ถาม:** ใช้ Python เวอร์ชันอะไร — ผมพัฒนาโดยใช้ Python 3 ร่วมกับ Django 6.0.6
- **ถาม:** ฐานข้อมูลตอนนี้ใช้อะไร — ตอนพัฒนาบนเครื่องใช้ SQLite บน Production ใช้ PostgreSQL
- **ถาม:** 1 หน้าแสดงสินค้ากี่ชิ้น — 6 ชิ้น และ API แบ่งหน้าละ 10
- **ถาม:** มีทั้งหมดกี่ตาราง — 11 โมเดล
- **ถาม:** Primary Key คืออะไร — ฟิลด์ id อัตโนมัติของทุกตาราง
- **ถาม:** มีกี่เทสต์ — 21 เทสต์ ผ่านทั้งหมด
- **ถาม:** บัญชีทดลอง — แอดมิน boss รหัส 1
- **ถาม:** คูปองมีอะไรบ้าง — WELCOME10 ลด 10% และ TOY20 ลด 20% ขั้นต่ำ 1,000 บาท